

MOVELINK

System-Dokumentation & Traceability-Report

Anforderungs- & Abdeckungsanalyse der MoveLink App

Status: Freigegeben

Sprache: Deutsch

Autoren: Erlind & AI Assistant Antigravity

Dokumentenversion: v1.1.0 (Scraped Live)

Veröffentlichungsdatum: 20. Juni 2026

1. Dokumente aus dem Projekt

In diesem Abschnitt werden die im Projekt gefundenen Markdown-Dateien im Volltext aufgeführt. Diese Dokumente bilden die Grundlage für die Anforderungen und Use Cases.

Datei: [doc/Requirements.md](#)

Systemanforderungen (Requirements)

Dieses Dokument definiert die funktionalen und nicht-funktionalen Anforderungen sowie die Randbedingungen des MoveLink-Systems.

Funktionale Anforderungen

FA1: Dashboard und Navigation

Das System muss eine Navigationsstruktur (Reiter/Menü) für Hardwaregeräte, Trainings und vergangene Trainingseinheiten bereitstellen. (Bezug: [UC-1](#), [UC-2](#), [UC-3](#))

FA2: Geräte-Scanning

Das System muss eine Liste verfügbarer Bluetooth-Hardwaregeräte anzeigen und den aktuellen Verbindungsstatus visualisieren. (Bezug: [UC-1](#))

FA3: Verbindungsaufbau

Das System muss in der Lage sein, eine stabile Bluetooth-Verbindung mit dem IMU-Sensor herzustellen. (Bezug: [UC-1](#))

FA4: Trainings-Detailansicht

Das System muss eine detaillierte Ansicht für ein ausgewähltes Training anzeigen. (Bezug: [UC-2](#))

FA5: Datenstrom-Verarbeitung

Das System muss kontinuierliche Bewegungsdatenströme vom Sensor empfangen, filtern und verarbeiten können. (Bezug: [UC-2](#))

FA6: Echtzeit-Visualisierung

Das System muss die empfangenen Sensordaten und Bewegungen in Echtzeit visualisieren. (Bezug: [UC-2](#))

FA7: Historische Analyse

Das System muss historische Bewegungsdaten grafisch und statistisch anzeigen können. (Bezug: **UC-3**)

Nicht-funktionale Anforderungen (Muss-Kriterien)

NF1: Latenz

Die End-to-End-Latenz von der physischen Sensorbewegung bis zur visuellen Darstellung in der App muss ≤ 100 ms sein.

NF2: Zuverlässigkeit und Reconnect

Bei Verbindungsabbrüchen muss die App den Nutzer umgehend benachrichtigen und automatische Wiederverbindungsversuche (Reconnect) starten.

NF3: Benutzbarkeit (Usability)

Das Bluetooth-Pairing mit dem Sensor darf maximal zwei manuelle Interaktionen erfordern.

Randbedingungen

R1: Plattform-Kompatibilität

Die mobile Applikation muss nativ oder als hybride App auf Android-Geräten lauffähig sein.

Datei: **doc/UseCases.md**

Anwendungsfälle (Use Cases)

Hier werden die primären Interaktionen zwischen dem Trainierenden und dem MoveLink-System beschrieben.

UC-1: Trainingsgerät verbinden

- **Akteur:** Trainierender
- **Vorbedingung:** Trainingsgerät ist eingeschaltet und befindet sich in Reichweite.
- **Beschreibung:** Als Trainierender möchte ich mein Trainingsgerät mit der App verbinden, um Trainingsdaten erfassen zu können.

- **Ablauf (Szenario):**

1. **Eingabe:** Der Trainierende öffnet die App.
- **Ausgabe*:** Die App zeigt eine Möglichkeit/einen Reiter für Hardwaregeräte an.
2. **Eingabe:** Der Trainierende klicke auf den Reiter "Hardwaregeräte".
- **Ausgabe*:** Die App zeigt eine Liste der verfügbaren Hardwaregeräte sowie den aktuellen Verbindungsstatus an.
3. **Eingabe:** Der Trainierende wählt ein Hardwaregerät aus der Liste aus und klickt auf "Verbinden".
- **Ausgabe*:** Die App zeigt die Detailansicht des ausgewählten Geräts und bestätigt den erfolgreichen Verbindungsaufbau.
-

UC-2: Echtzeit-Training überwachen

- **Akteur:** Trainierender
 - **Vorbedingung:** Das Trainingsgerät ist erfolgreich mit der App verbunden.
 - **Beschreibung:** Ich als Trainierender möchte mein Training in Echtzeit überwachen können, um direkt Feedback zu meiner Ausführung zu erhalten.
 - **Ablauf (Szenario):**
 1. **Eingabe:** Der Trainierende öffnet die App.
 - **Ausgabe*:** Die App zeigt die Option zum Starten eines Trainings an.
 2. **Eingabe:** Der Trainierende klickt auf "Training starten".
 - **Ausgabe*:** Die App zeigt die Detailansicht des ausgewählten Trainings sowie die Start-Schaltfläche.
 3. **Eingabe:** Der Trainierende drückt den "Start"-Button.
 - **Ausgabe*:** Die App demonstriert grafisch die auszuführende Übungsbewegung.
 4. **Eingabe:** Der Trainierende führt die Bewegung aus.
 - **Ausgabe*:** Die App visualisiert die Bewegung in Echtzeit, vergleicht sie mit der Zielvorgabe und gibt positives Feedback bei korrekter Ausführung.
-

UC-3: Trainingsdaten einsehen

- **Akteur:** Trainierender
 - **Vorbedingung:** Mindestens eine aufgezeichnete Trainingseinheit ist in der Datenbank vorhanden.
 - **Beschreibung:** Ich als Trainierender möchte vergangene Trainingseinheiten einsehen können, um meine Fortschritte zu verfolgen.
 - **Ablauf (Szenario):**
 1. **Eingabe:** Der Trainierende öffnet die App.
 - **Ausgabe*:** Die App bietet eine Option zum Einsehen des Trainingsverlaufs.
 2. **Eingabe:** Der Trainierende navigiert zum Reiter "Trainingseinheiten".
 - **Ausgabe*:** Die App listet alle vergangenen Trainingseinheiten chronologisch auf.
-

3. **Eingabe:** Der Trainierende wählt eine Trainingseinheit aus der Liste aus.

- **Ausgabe*:** Die App bereitet die historischen Bewegungsdaten grafisch und statistisch auf.

Datei: [app/architecture.md](#)

MoveLink Mobile App - Container-Architektur

Dieses Dokument beschreibt die Mobile App als eigenständige, deploybare Einheit im C4-Modell.

C4-Architektur-Ebene

- **C4-Ebene:** Container
- **Deployable:** Ja
- **Deployment-Artefakt:** Android Package (.apk) / iOS IPA
- **Technologie-Stack:** React Native, Expo, TypeScript, Zustand, BLE PLX

Beschreibung

Die MoveLink Mobile App ist die primäre Benutzerschnittstelle des Systems. Sie läuft auf Android- und iOS-Endgeräten und verbindet sich über Bluetooth Low Energy (BLE) mit dem embedded Sensor-Gerät, um Bewegungsdaten in Echtzeit zu erfassen, zu visualisieren und zur persistenten Speicherung an das Backend zu übertragen.

flowchart TD

```
User[Trainierender] -->|Interagiert mit| App[React Native App Container]
App -->|BLE Bluetooth| Sensor[Sensor Firmware Container]
App -->|REST/WebSockets| Backend[Backend API Container]
```

Komponenten in diesem Container

Die App enthält mehrere Komponenten (C4-Komponenten-Ebene):

1. **SideNav:** Navigationskomponente für die App-Steuerung. (Erfüllt: **FA1**)
2. **SensorCard:** Verwaltung der BLE-Geräteverbindung und Pairing. (Erfüllt: **FA2, FA3, NF3**)
3. **LiveChart:** Echtzeit-Visualisierung der IMU-Beschleunigungs- und Gyroskopwerte. (Erfüllt: **FA6**)
4. **SessionCard:** Visualisierung historischer Trainingseinheiten. (Erfüllt: **FA7**)
5. **BLE-Hook (useBLE):** Kapselt die Bluetooth-Gerätekommunikation und den Reconnect. (Erfüllt: **FA3, FA5, NF2**)

Datei: app/components/ProfileCard/architecture.md

Profil-Karte (ProfileCard)

Diese Komponente zeigt die Profildetails des angemeldeten Benutzers an.

C4-Architektur-Ebene

- **C4-Ebene:** Component
- **Deployable:** Nein (Läuft als Teil des Mobile App Containers)

Datenfluss

```

flowchart LR
    JWT[Authentifizierungs-Token] -->|1. User-ID auslesen| Controller[ProfileController]
    Controller -->|2. Query| DB[(Datenbank)]
    DB -->|3. Rohdaten| Controller
    Controller -->|4. Bereinigtes Profil DTO| Client[ProfileCard UI]
  
```

Datei: app/components/architecture.md

App Komponenten

Diese Verzeichnis enthält die UI-Komponenten der Mobile-/Web-Anwendung (z.B. Visualisierungen, Charts, Navigation).

C4-Architektur-Ebene

- **C4-Ebene:** Component
- **Deployable:** Nein (Läuft als Teil des Mobile App Containers)

Datenfluss in UI-Komponenten

Die Komponenten erhalten Daten über Props oder den globalen Store und rendern diese reaktiv.

```

flowchart LR
    Store[Globaler Store] -->|Reaktive Updates| SensorCard[SensorCard / LiveChart]
    SensorCard -->|Klick / Aktion| Actions[Store Actions]
    Actions -->|Dispatch| Store
  
```

- **LiveChart:** Rendert eintreffende Datenpunkte in Echtzeit.
- **SensorCard:** Zeigt den aktuellen Status und Verbindungszustand von Bluetooth-Geräten.

Datei: [doc/AI_DOCUMENTATION_GUIDE.md](#)

Leitfaden für KI-Dokumentation & Traceability (MoveLink)

Dieses Repository verwendet ein automatisiertes, integriertes Dokumentations- und Traceability-System. Es scannt Markdown-Dokumente und Quellcode-Dateien, um eine interaktive Weboberfläche (HTML Dashboard) sowie einen kompilierten PDF-Bericht zu generieren.

Damit zukünftige KIs und Entwickler neue Anforderungen, Use Cases und Architekturentwicklungen richtig dokumentieren, müssen die folgenden Standards eingehalten werden.

1. Definition von Anforderungen & Use Cases (.md-Dateien)

Alle Systemdefinitionen werden in Markdown-Dateien im Ordner doc/ gepflegt (z. B. doc/Requirements.md und doc/UseCases.md).

ID-Format & Konventionen

Jeder Eintrag muss eine eindeutige ID besitzen:

- UC-X: Use Cases (z. B. **UC-1**)
- FA-X: Funktionale Anforderungen (z. B. **FA1**)
- NF-X: Nicht-funktionale Anforderungen (z. B. **NF1**)
- R-X: Randbedingungen (z. B. **R1**)

Format der Deklaration in Markdown

Damit der Scraper (scrape_docs.py) die Einträge korrekt parsen kann, müssen sie in einer Zeile deklariert werden, gefolgt von einer Beschreibung:

****UC-1: Live-Ansicht der Übungen****

Dies ist die Beschreibung des Use Cases. Hier steht detaillierter Text, der auch über mehrere Zeilen gehen kann.

****FA2: Bluetooth LE Signalstärke****

Das System muss die BLE-Signalstärke des Sensors in Echtzeit ausgeben.

Verknüpfung zwischen Anforderungen und Use Cases (Traceability)

Um Anforderungen mit Use Cases zu verknüpfen, muss die Use-Case-ID in Klammern oder als Text in der Zeile der Anforderung stehen. Der Scraper sucht nach Querverweisen (z. B. (**UC-1**)):

****FA3: BLE Verbindungsaufbau (UC-1)****

Das System muss eine Bluetooth Low Energy Verbindung zum Sensor herstellen.

2. Implementierungs-Referenzen im Quellcode (@implements)

Um nachzuweisen, dass eine Anforderung tatsächlich im Code implementiert wurde, müssen Entwickler und KIs direkt in den Quellcodedateien (.ts, .tsx, .ino, .cpp, .h) @implements-Annotationen in Kommentaren hinzufügen.

Syntax

```
@implements ID1, ID2, ...
```

Code-Beispiele

- *In TypeScript / TSX Dateien (app/):**

```
// @implements FA2, FA3, NF2
export function SensorCard() {
  // UI Code...
}
```

- *In Arduino / C++ Dateien (embedded/):**

```
/*
 * @implements FA5, NF1
 * Liest Sensorwerte mit 50Hz aus und wendet einen Tiefpassfilter an.
 */
void loop() {
  // Sensorsignal...
}
```

3. C4-Architektur-Modellierung (Metadaten-Blöcke)

Jedes Architektur-Dokument in Markdown muss Informationen über die zugehörige C4-Ebene und die Deployability enthalten.

Metadaten-Header in Markdown

Fügen Sie ganz oben in der entsprechenden Architektur-Markdown-Datei (z. B. `app/architecture.md`) einen HTML-Kommentarblock mit folgenden Keys hinzu:

```
<!--
C4-Ebene: Container
Deployable: Ja
-->
```

- *Erlaubte Werte:**
- **C4-Ebene:** System-Context, Container, Component
- **Deployable:** Ja / Nein (oder Yes / No)

Registrierung im C4 Model Explorer

Wenn ein neuer Container oder eine neue Komponente hinzugefügt wird, muss diese auch in der C4_DATA-Struktur am Ende von `docs_site/app.js` registriert werden:

1. Tragen Sie das Element unter `containers.elements` oder `components.[container_id].elements` ein.
2. Definieren Sie dessen Verbindungen (Connectoren) im zugehörigen `connections`-Array.
3. Ergänzen Sie die Dateizuordnung in der Funktion `getC4ElementForFile` in `docs_site/app.js`, damit die E2E-Flussdiagramme die Datei dem neuen C4-Element zuweisen.

4. Build-Prozess & Pipeline

Nach jeder Änderung an der Dokumentation oder den `@implements`-Kommentaren im Quellcode müssen die Kompilierungsskripte ausgeführt werden:

Lokaler Build-Befehl

1. **Scraper ausführen** (erstellt `docs_site/data.js`):

```
`bash
```

```
python scrape_docs.py
```

```
`
```

2. **PDF Bericht generieren** (erstellt `docs_site/documentation_report.pdf`):

```
`bash
```

```
python generate_pdf.py
```

```
`
```

CI/CD Pipeline (GitHub Actions)

Bei jedem Push auf den main-Branch baut die Pipeline `.github/workflows/docs.yml` die Webseite und das PDF automatisch. Wenn Sie einen Commit pushen, der bereits kompilierte Änderungen enthält, nutzen Sie `[skip ci]` im Commit-Betreff, um endlose Build-Loops zu verhindern.

Datei: doc/firstnotes.md

Mikrocontroller <--- Datenpakete ---> App

Backend <- API -> Frontend

(Backend?)

Problemstellung:

Darstellung von Trainingsdatensätzen zur Bewegungsanalyse.

Projektebene

Stakeholder:

- Trainierenden (Fokus)
- Entwickler
- Dienstleister
- Hochschule

Anforderungen an die App:

2 Zustände:

1 Zustand (Fokus) aktives Training:

- Intention: visuelle Darstellung des Trainingsfortschritts in Echtzeit
- FA: In Echtzeit eine Bewertung abliefern
- FA: Darstellung der Ausführung
- NFA: Eine eigenständige Applikation
- NFA: Das Darstellen der darf Höchstens 1 Sekunden dauern
- Rahmenbedingung: Auf Android, Mac und Windows funktionsfähig
- Technologien: Flutter & MAUI

Ich habe einen XIOA nRF52840 Gerät. Ich möchte die Daten in Echtzeit in einem Frontend anzeigen z.B. React. Wie sinnvoll und komplex wäre es einen Server dazwischen zu packen?

Datei: [embedded/architecture.md](#)

MoveLink Embedded Firmware - Container-Architektur

Dieses Dokument beschreibt die Embedded Sensor-Firmware als eigenständige, deploybare Einheit im C4-Modell.

C4-Architektur-Ebene

- **C4-Ebene:** Container
- **Deployable:** Ja
- **Deployment-Artefakt:** Binär-Firmware (flashed via USB/Serial)
- **Technologie-Stack:** Arduino C/C++, LSM6DS3 IMU Library, Edge Impulse SDK, Bluetooth Low Energy

Beschreibung

Die Sensor-Firmware läuft auf dem XIAO nRF52840 Sense Controller. Sie erfasst Beschleunigungs- und Rotationsdaten über den integrierten LSM6DS3-Sensor mit einer festen Abtastrate (50Hz), wendet Signalfilterungen zur Rauschunterdrückung an und streamt die Datenpakete als binäres Array via BLE Characteristics an die Mobile App. Alternativ führt sie Edge-Impulse-Inferenzmodelle direkt auf dem Mikrocontroller aus, um Trainingsübungen (z.B. Bizeps-Curls) lokal zu klassifizieren und Fehler über die integrierten RGB-LEDs anzuzeigen.

flowchart TD

```
Sensor[MPU6050/LSM6DS3 IMU Sensor] -->|I2C Rohdaten| Firmware[XIAO nRF52840 Arduino Firmware]
Firmware -->|Inferenz / Signalfilterung| BLE[BLE Characteristic]
BLE -->|Paket-Stream| App[React Native Mobile App]
```

Komponenten in diesem Container

1. **Sensordatenerfassung (Loop):** Liest kontinuierlich Beschleunigung (X, Y, Z) und Gyroskop (X, Y, Z). (Erfüllt: **FA5**)
2. **Inferenz-Engine (Edge Impulse):** Klassifiziert Übungsausführungen lokal auf dem Chip. (Erfüllt: **FA5**)
3. **LED- & Display-Controller:** Bietet direktes visuelles Feedback an den Nutzer bei Fehlern. (Erfüllt: **FA5**)

Datei: [embedded/src/architecture.md](#)

Embedded Sensor-Firmware

Die Firmware liest Sensorwerte aus und überträgt diese über Bluetooth Low Energy (BLE) an die App.

Datenfluss Firmware

flowchart TD

```

    Sensor[MPU6050 Beschleunigungssensor] -->|I2C Rohdaten| Arduino[Arduino / ESP32 Controller]
    Arduino -->|Signalfilterung & Skalierung| BLE[Bluetooth Low Energy Characteristic]
    BLE -->|Notifikationen / Byte-Array| App[Mobile App]
  
```

- **Sensordatenerfassung:** Erfolgt in einer festen Frequenz (z.B. 50Hz).
- **Filterung:** Tiefpassfilter zur Rauschminderung auf dem Mikrocontroller.
- **BLE-Transfer:** Effiziente Übertragung als binäres Datenpaket.

2. Requirements vs. Use Cases Matrix

Die folgende Matrix veranschaulicht die Beziehungen zwischen den definierten funktionalen Anforderungen (FA) / nicht-funktionalen Anforderungen (NF) / Rahmenbedingungen (R) und den Use Cases (UC).

Anforderung / ID	UC-1	UC-2	UC-3
FA1: Dashboard und Navigation Das System muss eine...	X	X	X
FA2: Bluetooth LE Signalstärke** Das System muss d...	-	-	-
FA3: BLE Verbindungsaufbau (UC-1)** Das System mus...	X	-	-
FA4: Trainings-Detailansicht Das System muss eine ...	-	X	-
FA5: Datenstrom-Verarbeitung Das System muss konti...	-	X	-
FA6: Echtzeit-Visualisierung Das System muss die e...	-	X	-
FA7: Historische Analyse Das System muss historisc...	-	-	X
NF1: Latenz Die End-to-End-Latenz von der physisch...	-	-	-
NF2: Zuverlässigkeit und Reconnect Bei Verbindungs...	-	-	-
NF3: Benutzbarkeit (Usability) Das Bluetooth-Pairi...	-	-	-
R1: Plattform-Kompatibilität Die mobile Applikati...	-	-	-

Hinweis: Ein 'X' kennzeichnet eine direkte Verknüpfung im Pflichtenheft bzw. den Anforderungsdokumenten.

3. End-to-End Rückverfolgbarkeits-Baum

Hier ist der vollständige Trace-Pfad von den Anwendungsfällen über die funktionalen Anforderungen bis hin zu den konkreten Code-Implementierungen (z. B. React Native-Dateien, Arduino-Skripte) dargestellt.

Anwendungsfall UC-1: Trainingsgerät verbinden

- **Anforderung FA1:** Dashboard und Navigation Das System muss eine Navigationsstruktur (Reiter/Menü) für Hardwaregeräte, Trainings und vergangene Trainingseinheiten bereitstellen. *(Bezug: UC-1, UC-2, UC-3)*

- **Implementiert in:** doc/Pflichtenheft/pflichtenheft.tex (Zeile 217)

Kontext: \item \textbf{FA1:} Das System muss einen Reiter oder eine Navigationsmöglichkeit für Hardwaregeräte / Trainings / vergangene Trainingseinheiten anzeigen. (aus UC-1, UC-2, UC-3)

- **Implementiert in:** app/app/(tabs)/_layout.tsx (Zeile 2)

Kontext: * @implements FA1

- **Implementiert in:** doc/AI_DOCUMENTATION_GUIDE.md (Zeile 16)

Kontext: * **FA-X**: Funktionale Anforderungen (z. B. **FA1**)

- **Implementiert in:** app/architecture.md (Zeile 23)

Kontext: 1. **SideNav**: Navigationskomponente für die App-Steuerung. (Erfüllt: FA1)

- **Anforderung FA3:** BLE Verbindungsaufbau (UC-1)** Das System muss eine Bluetooth Low Energy Verbindung zum Sensor herstellen. ``

- **Implementiert in:** doc/Pflichtenheft/pflichtenheft.tex (Zeile 219)

Kontext: \item \textbf{FA3:} Das System muss einen Verbindungsaufbau mit dem Hardwaregerät herstellen können. (aus UC-1)

- **Implementiert in:** app/components/SensorCard.tsx (Zeile 2)

Kontext: * @implements FA2, FA3, NF3

- **Implementiert in:** app/hooks/useBLE.ts (Zeile 2)

Kontext: * @implements FA3, FA5, NF2

- **Implementiert in:** app/app/(tabs)/index.tsx (Zeile 2)

Kontext: * @implements FA2, FA3, FA4, FA5, FA6

- **Implementiert in:** doc/Requirements.md (Zeile 15)

Kontext: **FA3**: Verbindungsaufbau

- **Implementiert in:** doc/AI_DOCUMENTATION_GUIDE.md (Zeile 54)

Kontext: // @implements FA2, FA3, NF2

- **Implementiert in:** app/architecture.md (Zeile 24)

Kontext: 2. **SensorCard**: Verwaltung der BLE-Geräteverbindung und Pairing. (Erfüllt: FA2, FA3, NF3)

- **Implementiert in:** app/architecture.md (Zeile 27)

Kontext: 5. **BLE-Hook (useBLE)**: Kapselt die Bluetooth-Gerätekommunikation und den Reconnect. (Erfüllt: FA3, FA5, NF2)

Anwendungsfall UC-2: Echtzeit-Training überwachen

- **Anforderung FA1:** Dashboard und Navigation Das System muss eine Navigationsstruktur (Reiter/Menü) für Hardwaregeräte, Trainings und vergangene Trainingseinheiten bereitstellen. *(Bezug: UC-1, UC-2, UC-3)*

- **Implementiert in:** doc/Pflichtenheft/pflichtenheft.tex (Zeile 217)

Kontext: \item \textbf{FA1:} Das System muss einen Reiter oder eine Navigationsmöglichkeit für Hardwaregeräte / Trainings / vergangene Trainingseinheiten anzeigen. (aus UC-1, UC-2, UC-3)

- **Implementiert in:** app/app/(tabs)/_layout.tsx (Zeile 2)

Kontext: * @implements FA1

- **Implementiert in:** doc/AI_DOCUMENTATION_GUIDE.md (Zeile 16)

Kontext: * ``FA-X``: Funktionale Anforderungen (z. B. ``FA1``)

- **Implementiert in:** app/architecture.md (Zeile 23)

Kontext: 1. **SideNav**: Navigationskomponente für die App-Steuerung. (Erfüllt: FA1)

- **Anforderung FA4:** Trainings-Detailansicht Das System muss eine detaillierte Ansicht für ein ausgewähltes Training anzeigen. *(Bezug: UC-2)*

- **Implementiert in:** doc/Pflichtenheft/pflichtenheft.tex (Zeile 220)

Kontext: \item \textbf{FA4:} Das System muss eine Detailansicht für ein ausgewähltes Training anzeigen. (aus UC-2)

- **Implementiert in:** app/app/(tabs)/index.tsx (Zeile 2)

Kontext: * @implements FA2, FA3, FA4, FA5, FA6

- **Anforderung FA5:** Datenstrom-Verarbeitung Das System muss kontinuierliche Bewegungsdatenströme vom Sensor empfangen, filtern und verarbeiten können. *(Bezug: UC-2)*

- **Implementiert in:** embedded/src/Executable.ino (Zeile 2)

Kontext: * @implements FA5

- **Implementiert in:** embedded/src/Training_Skript/Training_Skript.ino (Zeile 2)

Kontext: // @implements FA5

- **Implementiert in:** doc/Pflichtenheft/pflichtenheft.tex (Zeile 221)

Kontext: \item \textbf{FA5:} Das System muss Bewegungsdatenströme empfangen und verarbeiten können. (aus UC-2)

- **Implementiert in:** app/store/index.ts (Zeile 2)

Kontext: * @implements FA5

- **Implementiert in:** app/hooks/useBLE.ts (Zeile 2)

Kontext: * @implements FA3, FA5, NF2

- **Implementiert in:** app/hooks/useWebSocket.ts (Zeile 2)

Kontext: * @implements FA5

- **Implementiert in:** app/app/(tabs)/index.tsx (Zeile 2)

Kontext: * @implements FA2, FA3, FA4, FA5, FA6

- **Implementiert in:** embedded/architecture.md (Zeile 22)

Kontext: 1. **Sensordatenerfassung (Loop)**: Liest kontinuierlich Beschleunigung (X, Y, Z) und Gyroskop (X, Y, Z). (Erfüllt: FA5)

- **Implementiert in:** embedded/architecture.md (Zeile 23)

Kontext: 2. **Inferenz-Engine (Edge Impulse)**: Klassifiziert Übungsausführungen lokal auf dem Chip.

(Erfüllt: FA5)

- **Implementiert in:** `embedded/architecture.md` (Zeile 24)

Kontext: 3. ****LED- & Display-Controller****: Bietet direktes visuelles Feedback an den Nutzer bei Fehlern. (Erfüllt: FA5)

- **Implementiert in:** `doc/AI_DOCUMENTATION_GUIDE.md` (Zeile 63)

Kontext: * @implements FA5, NF1

- **Implementiert in:** `app/architecture.md` (Zeile 27)

Kontext: 5. ****BLE-Hook (useBLE)****: Kapselt die Bluetooth-Gerätekommunikation und den Reconnect. (Erfüllt: FA3, FA5, NF2)

- **Anforderung FA6:** Echtzeit-Visualisierung Das System muss die empfangenen Sensordaten und Bewegungen in Echtzeit visualisieren. *(Bezug: UC-2)*

- **Implementiert in:** `doc/Pflichtenheft/pflichtenheft.tex` (Zeile 222)

Kontext: \item \textbf{FA6:} Das System muss die empfangenen Bewegungen in Echtzeit visualisieren. (aus UC-2)

- **Implementiert in:** `app/components/LiveChart.tsx` (Zeile 2)

Kontext: * @implements FA6

- **Implementiert in:** `app/app/(tabs)/index.tsx` (Zeile 2)

Kontext: * @implements FA2, FA3, FA4, FA5, FA6

- **Implementiert in:** `app/architecture.md` (Zeile 25)

Kontext: 3. ****LiveChart****: Echtzeit-Visualisierung der IMU-Beschleunigungs- und Gyroskopwerte. (Erfüllt: FA6)

Anwendungsfall UC-3: Trainingsdaten einsehen

- **Anforderung FA1:** Dashboard und Navigation Das System muss eine Navigationsstruktur (Reiter/Menü) für Hardwaregeräte, Trainings und vergangene Trainingseinheiten bereitstellen. *(Bezug: UC-1, UC-2, UC-3)*

- **Implementiert in:** `doc/Pflichtenheft/pflichtenheft.tex` (Zeile 217)

Kontext: \item \textbf{FA1:} Das System muss einen Reiter oder eine Navigationsmöglichkeit für Hardwaregeräte / Trainings / vergangene Trainingseinheiten anzeigen. (aus UC-1, UC-2, UC-3)

- **Implementiert in:** `app/app/(tabs)/_layout.tsx` (Zeile 2)

Kontext: * @implements FA1

- **Implementiert in:** `doc/AI_DOCUMENTATION_GUIDE.md` (Zeile 16)

Kontext: * ****FA-X****: Funktionale Anforderungen (z. B. **FA1**)

- **Implementiert in:** `app/architecture.md` (Zeile 23)

Kontext: 1. ****SideNav****: Navigationskomponente für die App-Steuerung. (Erfüllt: FA1)

- **Anforderung FA7:** Historische Analyse Das System muss historische Bewegungsdaten grafisch und statistisch anzeigen können. *(Bezug: UC-3)*

- **Implementiert in:** `doc/Pflichtenheft/pflichtenheft.tex` (Zeile 223)

Kontext: \item \textbf{FA7:} Das System muss historische Bewegungsdaten grafisch anzeigen können. (aus UC-3)

- **Implementiert in:** app/components/SessionCard.tsx (Zeile 2)

Kontext: * @implements FA7

- **Implementiert in:** app/app/(tabs)/history.tsx (Zeile 2)

Kontext: * @implements FA7

- **Implementiert in:** app/architecture.md (Zeile 26)

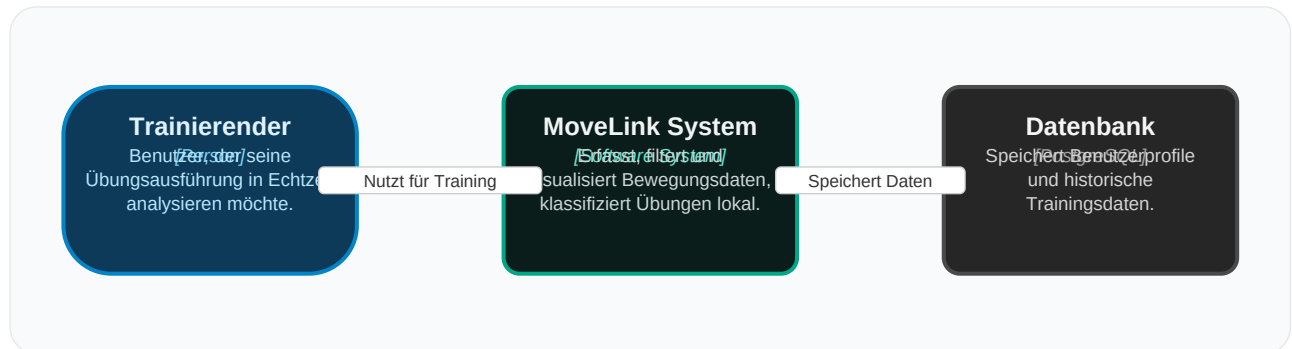
Kontext: 4. ****SessionCard****: Visualisierung historischer Trainingseinheiten. (Erfüllt: FA7)

4. System-Architektur (C4 Modell)

In diesem Abschnitt wird die Systemarchitektur auf den verschiedenen C4-Ebenen (System-Kontext, Container und Komponenten) visualisiert.

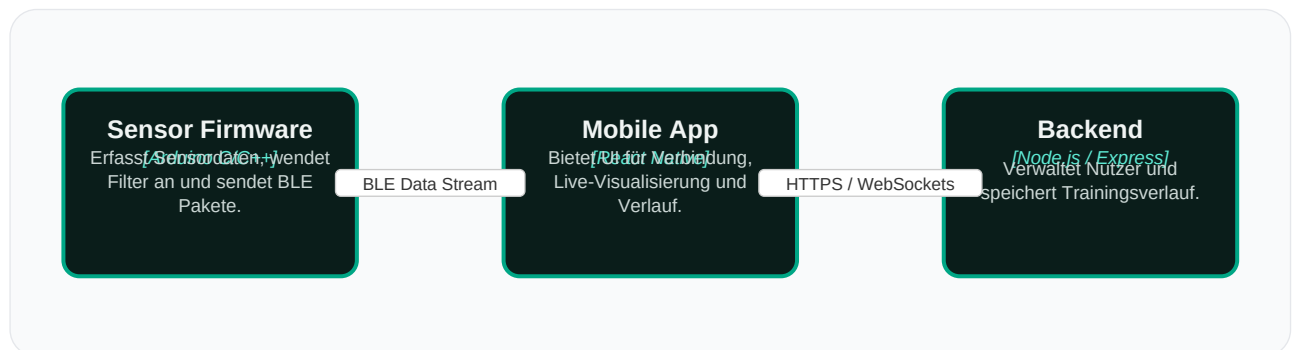
4.1 System-Kontext-Diagramm

Das System-Kontext-Diagramm zeigt die Position von MoveLink in Bezug auf Benutzer und das externe Datenbanksystem.



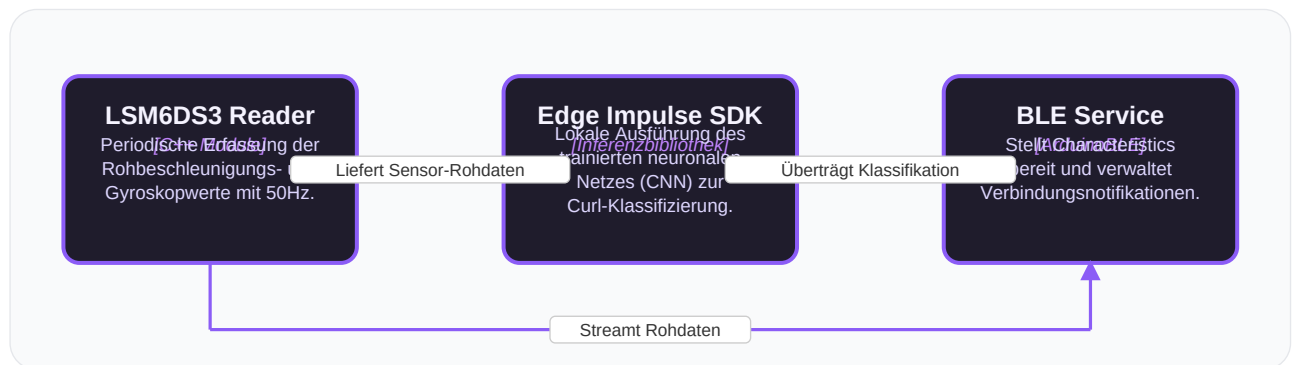
4.2 Container-Diagramm

Das Container-Diagramm veranschaulicht die Aufteilung des MoveLink Systems in eigenständige, ausführbare Subsysteme (Mobile App, Firmware und Backend) sowie deren Kommunikationspfade.



4.3 Komponenten-Diagramm (Sensor-Firmware)

Das Komponenten-Diagramm zeigt das Innenleben der Sensor-Firmware, die auf dem Xiao-Mikrocontroller läuft, sowie die Kopplung von Sensorik, Inferenz (Edge Impulse) und Bluetooth BLE.



4.4 Komponenten-Diagramm (Mobile App)

Dieses Diagramm zeigt die internen Komponenten des React Native Containers (Mobile App), aufgeteilt in UI-Elemente und Custom Hooks, die mit BLE und dem WebSocket-Server interagieren.

